

JAVA

Abstração em Java

Classes Abstratas • Interfaces • JDK 8/9 • Polimorfismo

Apostila completa com explicações detalhadas,
exemplos práticos e mini-projects guiados.

CONTEÚDO DA APOSTILA

- 01 Introdução e Conceitos Fundamentais
- 02 Classes Abstratas
- 03 Interfaces — Fundamentos
- 04 Interfaces — Avançado
- 05 Novidades JDK 8 e JDK 9
- 06 Interface vs. Classe Abstrata
- 07 Mini-Projects Práticos
- 08 Boas Práticas e Padrões

1

Introdução & Conceitos Fundamentais

Abstração, Generalização e o suporte do Java a esses princípios

*Nesta apostila, exploramos dois dos conceitos mais poderosos da programação orientada a objetos: **abstração** e **generalização**. Dominar esses conceitos é o que separa um programador que escreve código funcional de um que escreve código extensível, reutilizável e fácil de manter.*

Generalização — identificando o que é comum

Generalizar é o ato de identificar características e comportamentos comuns entre diferentes objetos e agrupá-los sob um conceito mais amplo. No mundo real, fazemos isso constantemente: um cachorro, um gato e um pinguim são animais muito diferentes, mas todos compartilham certos atributos — eles se movem, fazem sons, têm idade, possuem um nome.

No design de software, esse processo se traduz na criação de **hierarquias de classes**. A classe mais alta na hierarquia representa o conceito mais geral — o bloco de construção fundamental que todos os subtipos compartilham. Quanto mais você desce na hierarquia, mais específico e concreto se torna o tipo.

Abstração — representando grupos, não instâncias

Abstração é parte natural do processo de generalização. Quando falamos de 'Animal' como conceito, não estamos pensando em nenhum animal específico — estamos representando um conjunto de características e comportamentos comuns a todos os animais. O conceito de 'Animal' é **abstrato**: não existe um objeto chamado 'Animal' no mundo real.

Em Java, a abstração se manifesta de duas formas principais: através das **classes abstratas** e das **interfaces**. Ambas permitem definir contratos de comportamento — o que um objeto deve ser capaz de fazer — sem especificar exatamente como ele fará. Uma regra prática simples: se você não consegue desenhar esse objeto em um papel, provavelmente é um conceito abstrato.

Do ponto de vista do Java: se você pode instanciar uma classe diretamente com `new`, ela é concreta. Se não pode — se o Java impede essa instanciação — é abstrata. Essa distinção é fundamental para o design correto de hierarquias.

Método Abstrato — o contrato de comportamento

Um **método abstrato** é declarado com a assinatura (nome, parâmetros e tipo de retorno) mas sem corpo — sem implementação. Ele descreve *o que* um objeto deve ser capaz de fazer, mas deixa *como* para as subclasses definirem. Pense nele como uma promessa: toda classe concreta que herdar

esse comportamento promete fornecer sua própria implementação.

```
public abstract void mover(); // sem corpo – apenas a assinatura

// Exemplo – Animal abstrato força subclasses a implementar:
abstract class Animal {
    abstract void mover();
    abstract void fazerSom();
}
```

2

Classes Abstratas

Declaração, herança, métodos abstratos e quando utilizá-las

Declarando uma Classe Abstrata

Para declarar uma classe abstrata em Java, basta adicionar a palavra-chave `abstract` antes de `class`. Isso instrui o compilador a impedir qualquer tentativa de instanciar essa classe diretamente com `new`. Ela só pode ser usada como base — outras classes a estendem e fornecem as implementações necessárias.

Uma classe abstrata pode ter **construtor** — que é chamado pelas subclasses via `super()`. Pode ter **campos de instância** (estado interno) — diferente das interfaces. Pode ter um mix de **métodos abstratos** (sem corpo) e **métodos concretos** (com corpo). Essa flexibilidade é a grande vantagem das classes abstratas sobre as interfaces para hierarquias próximas.

```
// Classe abstrata – não pode ser instanciada com new Animal()
abstract class Animal {
    private String nome;
    private int idade;

    public Animal(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    // Métodos abstratos – obrigatórios nas subclasses:
    public abstract void mover();
    public abstract void fazerSom();

    // Método concreto – compartilhado por todas as subclasses:
    public String getNome() { return nome; }
}

// ■ Uma classe abstrata PODE ter construtor – chamado via super()
```

Hierarquias com Classes Abstratas

Uma subclasse de uma classe abstrata pode ser ela mesma abstrata — não precisa implementar todos os métodos abstratos herdados. Apenas quando a hierarquia chega a uma classe concreta (sem `abstract`) é que todos os métodos abstratos pendentes devem ser implementados. O compilador garante isso em tempo de compilação.

```
abstract class Animal {
    abstract void mover();
    abstract void fazerSom();
}

abstract class Mamifero extends Animal {
    abstract void perderPelo();
    // mover() e fazerSom() ainda precisam ser implementados
}

class Cachorro extends Mamifero {
    @Override public void mover() { System.out.println("Cachorro correndo"); }
    @Override public void fazerSom() { System.out.println("Au au!"); }
    @Override public void perderPelo() { System.out.println("Perdendo pelo..."); }
    // Todos os métodos abstratos implementados – Cachorro é concreta!
}
```

■ Quando usar Classe Abstrata?

Use classe abstrata quando subclasses compartilham ESTADO (campos como nome, idade) e COMPORTAMENTO PARCIAL (métodos concretos comuns). A classe abstrata fornece o esqueleto; as subclasses preenchem os detalhes específicos.

3**Interfaces — Fundamentos**

Declaração, implements, modificadores implícitos e constantes

O que é uma Interface?

Uma **interface** em Java é um tipo especial — não é uma classe. Ela define um **contrato puro de comportamento**: especifica o que uma classe deve ser capaz de fazer, sem nenhuma informação sobre como ou com que estado interno. Enquanto uma classe abstrata diz '*o que você é*' (relação is-a), uma interface diz '*o que você pode fazer*' (can-do).

A grande vantagem das interfaces sobre as classes abstratas é a **múltipla implementação**: uma classe pode implementar quantas interfaces quiser, mas só pode estender uma única classe. Isso permite que objetos completamente diferentes — um Jato, um Pássaro, uma Libélula — sejam tratados uniformemente como `FlightEnabled`, simplesmente porque todos implementam os mesmos métodos de voo.

Declarando e Usando Interfaces

Use a palavra-chave `interface` no lugar de `class`. Uma convenção comum do Java é nomear interfaces com sufixo `-able` (`Comparable`, `Serializable`, `Iterable`), indicando uma capacidade. A classe que implementa a interface usa `implements` e deve fornecer corpo para todos os métodos abstratos da interface.

```
// Declarando interfaces:
public interface FlightEnabled {
    void decolar(); // implicitamente public abstract
    void voar();
    void pousar();
}

public interface Rastreavel {
    void rastrear();
}

// Implementando múltiplas interfaces:
class Passaro extends Animal implements FlightEnabled, Rastreavel {
    @Override public void mover() { System.out.println("Passaro se move"); }
    @Override public void decolar() { System.out.println("Passaro decolando!"); }
    @Override public void voar() { System.out.println("Passaro voando!"); }
    @Override public void pousar() { System.out.println("Passaro pousando!"); }
    @Override public void rastrear() { System.out.println("Rastreando passaro..."); }
}

// Polimorfismo de interface:
FlightEnabled voador = new Passaro(); // válido!
```

Modificadores Implícitos e Constantes

Interfaces têm regras especiais sobre modificadores. Todo método declarado sem corpo em uma interface é **implicitamente** `public abstract` — você não precisa escrever esses modificadores, mas eles estão lá. Em classes comuns, membros sem modificador de acesso são *package-private* — em interfaces, são `public`. Essa é uma diferença crucial que costuma confundir iniciantes.

Campos declarados em interfaces são sempre `public static final` — ou seja, constantes da interface. A convenção é nomeá-las em **ALL_CAPS_WITH_UNDERSCORES**. Elas são acessadas via o nome da interface: `FlightEnabled.VELOCIDADE_MAX`.

```
public interface FlightEnabled {
    // Todas as formas abaixo são equivalentes:
    int VELOCIDADE_MAX = 400;
    public static final int VELOCIDADE_MAX = 400; // mais explícito

    // Métodos abstratos (implicitamente public abstract):
    void decolar();
    void voar();
}

// Acesso à constante:
System.out.println(FlightEnabled.VELOCIDADE_MAX); // 400
```

4

Interfaces — Avançado

Herança de interfaces, polimorfismo e coding to an interface

Polimorfismo com Interfaces — agrupando pelo comportamento

Uma das maiores vantagens das interfaces é a capacidade de tratar objetos completamente diferentes de forma uniforme, baseando-se apenas no comportamento que eles compartilham. Um Jato, um Pássaro e uma Libélula não têm nenhum ancestral comum relevante — mas todos implementam `FlightEnabled`. Você pode colocá-los em um array e iterar, chamando `decolar()` em todos sem saber o tipo concreto de cada um.

```
FlightEnabled[] voadores = { new Passaro(), new Jato(), new Libelula() };

for (FlightEnabled v : voadores) {
    v.decolar(); // cada um executa sua própria implementação
    v.voar();
    v.pousar();
}
```

Herança de Interfaces

Assim como classes herdam de outras classes, interfaces podem **estender** outras interfaces — e podem estender *múltiplas* ao mesmo tempo (ao contrário das classes, que só podem estender uma). Uma interface filha herda todos os métodos abstratos das interfaces pai. Importante: interfaces usam `extends`, não `implements`.

```
// Interface pode estender múltiplas interfaces:
public interface SuperPoder extends FlightEnabled, Rastreavel {
    void usarPoder();
}

// INVÁLIDO – interface não usa implements:
// public interface Foo implements FlightEnabled { } // ERRO DE COMPILAÇÃO!
```

Coding to an Interface — a melhor prática

Coding to an Interface é o princípio de usar tipos de interface (ou abstratos) como o tipo da variável, parâmetro ou retorno de método, em vez do tipo concreto. Isso desacopla seu código da implementação específica, permitindo que você troque a implementação em um único lugar sem refatorar nada mais. É considerada uma das melhores práticas em Java.

```
// ■ Ruim – acoplado à implementação concreta:
ArrayList<String> lista = new ArrayList<>();

// ■ Melhor – programando para a interface:
List<String> lista = new ArrayList<>();
// Agora você pode trocar por LinkedList em um só lugar:
List<String> lista = new LinkedList<>(); // sem alterar mais nada!
```

5

Novidades JDK 8 e JDK 9

default methods, métodos estáticos públicos e métodos privados

O Método default — extensão retrocompatível

Antes do JDK 8, interfaces só podiam ter métodos abstratos. Isso criava um grande problema de extensibilidade: adicionar um novo método a uma interface quebrava imediatamente todas as classes que a implementavam — imagine 50 classes que precisariam ser atualizadas ao mesmo tempo.

O JDK 8 resolveu isso com o **método default**: um método com implementação dentro da interface. Ele é usado como 'implementação padrão' — classes que não o sobrescrevem herdam esse comportamento automaticamente. Classes existentes não precisam ser alteradas, tornando a mudança completamente retrocompatível.


```
public interface FlightEnabled {
    void decolar();
    void voar();
    void pousar();

    // Método default – concreto, pode ser sobrescrito:
    default void verificarSeguranca() {
        System.out.println("Verificacao de seguranca padrao OK");
    }
}

// Classes existentes NÃO precisam implementar verificarSeguranca()
// Retrocompatível – não quebra código já existente!
```

Sobrescrevendo Métodos default — 3 opções

Ao implementar uma interface com métodos default, você tem três opções para cada um deles: não sobrescrever (herda a implementação padrão), sobrescrever completamente com sua própria lógica, ou sobrescrever e também chamar a implementação da interface usando a sintaxe especial `NomeInterface.super.metodo()`.

```
// Opção 1: não sobrescrever – herda o default automaticamente

// Opção 2: sobrescrever completamente:
@Override
public void verificarSeguranca() {
    System.out.println("Verificacao customizada do Jato");
}

// Opção 3: chamar a default E adicionar lógica extra:
@Override
public void verificarSeguranca() {
    FlightEnabled.super.verificarSeguranca(); // chama a default
    System.out.println("+ Checagem extra de combustivel");
}
```

Métodos Estáticos (JDK 8) e Privados (JDK 9)

O JDK 8 também introduziu **métodos static** em interfaces — utilitários que pertencem à interface, não às instâncias. São acessados via `NomeInterface.metodo()` e são ideais para factory methods ou utilitários relacionados ao contrato da interface.

O JDK 9 deu mais um passo e permitiu **métodos private** em interfaces — tanto estáticos quanto de instância. Eles servem para evitar duplicação de código entre métodos default: você extrai a lógica

comum para um método privado que só os métodos default e outros private da mesma interface podem chamar.

```
public interface FlightEnabled {  
    // Método static – utilitário na interface:  
    static String criarRelatorio(FlightEnabled f) {  
        return "Voador: " + f.getClass().getSimpleName();  
    }  
  
    // Método private – reutilização interna (JDK 9):  
    private void logInterno(String msg) {  
        System.out.println("[LOG] " + msg);  
    }  
  
    default void verificarSeguranca() {  
        logInterno("verificando..."); // usa o private method  
    }  
}  
  
// Uso do método static:  
String relatorio = FlightEnabled.criarRelatorio(passaro);
```

6

Interface vs. Classe Abstrata

Comparação detalhada e orientações para escolha

Comparação detalhada

Campos de instância	■ Sim	■ Apenas constantes
Construtores	■ Sim	■ Não
Modificadores de acesso	■ Qualquer	■ Apenas public
Herança múltipla	■ Apenas uma classe	■ Múltiplas interfaces
Palavra-chave na classe	extends	implements
Métodos concretos	■ Sim	■ Apenas default/static/private
Relação semântica	is-a (é um)	can-do (pode fazer)

Quando usar cada um?

Escolha **classe abstrata** quando suas subclasses são intimamente relacionadas e precisam compartilhar estado (campos como nome, id, configuração) e código concreto. Quando você quer usar modificadores de acesso além de `public` nos métodos. Quando o relacionamento semântico é de herança (*'um Cachorro é um Animal'*).

Escolha **interface** quando classes sem nenhuma relação de herança precisam do mesmo comportamento. Quando você quer definir o contrato sem se preocupar com o estado. Quando uma classe precisa de múltiplos 'papéis' simultâneos (ex: um Pássaro que é `Animal`, `FlightEnabled` e `Rastreavel` ao mesmo tempo). Desde o JDK 8, interfaces com métodos default são ainda mais poderosas e flexíveis.

7

Mini-Projects

Loja com classe abstrata e sistema de mapeamento com interface

Os mini-projects a seguir aplicam os conceitos desta apostila em cenários realistas. O primeiro usa classe abstrata para modelar um sistema de loja genérico; o segundo usa interface para criar um sistema de mapeamento geoespacial. Ambos demonstram como abstração e polimorfismo trabalham juntos para criar código extensível.

Mini-Project 1 — Loja com Classe Abstrata

Construa um sistema de loja capaz de vender qualquer tipo de produto sem precisar ser reescrito. Crie a classe abstrata `ProdutoParaVenda` com os campos `tipo`, `preco` e `descricao`. Implemente os métodos concretos `getSalesPrice(qtd)` e `printPricedItem(qtd)`, que são iguais para todo produto. Declare o método abstrato `showDetails()`, que cada produto implementará de forma única.

Crie pelo menos 2 classes concretas (ex: `Livro`, `Eletronico`, `Roupa`) estendendo `ProdutoParaVenda`. Crie um record `OrderItem` com quantidade e produto. Na classe `Store`, monte uma lista de pedidos misturados e imprima o total.

```
abstract class ProdutoParaVenda {
    protected String tipo;
    protected double preco;
    protected String descricao;

    public double getSalesPrice(int qtd) { return qtd * preco; }

    public void printPricedItem(int qtd) {
        System.out.printf("%d x %-20s R$%.2f%n", qtd, tipo, getSalesPrice(qtd));
    }

    public abstract void showDetails(); // cada produto implementa do seu jeito
}

record OrderItem(int quantidade, ProdutoParaVenda produto) {}

class Livro extends ProdutoParaVenda {
    private String autor;
    @Override
    public void showDetails() { System.out.println(tipo + " por " + autor); }
}
```

Mini-Project 2 — Sistema de Mapas com Interface

Construa um sistema que representa dados geoespaciais como texto no formato GeoJSON. Crie a interface `Mappable` com os métodos `getLabel()`, `getShape()` (retorna um enum `Geometry`: `POINT`, `LINE` ou `POLYGON`) e `getMarker()`. Adicione uma constante `String JSON_PROPERTY = "properties":{ %s }`, um método default `String toJSON()` e um método static `void printFeature(Mappable m)`.

Crie as classes `Building` (geometria `POINT`) e `UtilityLine` (geometria `LINE`) implementando `Mappable`. Use enums para `Geometry`, `Color`, `PointMarker` e `LineMarker`. Demonstre o polimorfismo chamando `Mappable.printFeature()` com ambas as classes.

```
public interface Mappable {
    String JSON_PROPERTY = "\"properties\":{\"%s}\"";
    String getLabel();
    Geometry getShape();
    String getMarker();

    default String toJSON() {
        return String.format("{\"type\":\"%s\",\"label\":\"%s\"}", getShape(), getLabel());
    }

    static void printFeature(Mappable m) {
        System.out.println(String.format(JSON_PROPERTY, m.toJSON()));
    }
}

enum Geometry { POINT, LINE, POLYGON }

class Building implements Mappable { /* implementa os métodos */ }
class UtilityLine implements Mappable { /* implementa os métodos */ }

// Uso polimórfico:
Mappable.printFeature(new Building("Prefeitura", "publico"));
Mappable.printFeature(new UtilityLine("Energia", Color.YELLOW));
```

8

Boas Práticas & Padrões

Design patterns, erros comuns e orientações para código profissional

Erros Comuns — Evite!

```
// ■ Instanciar classe abstrata ou interface:
Animal a = new Animal();           // ERRO – Animal é abstrata!
FlightEnabled f = new FlightEnabled(); // ERRO – é interface!

// ■ Esquecer de implementar todos os métodos abstratos:
class Ave implements FlightEnabled {
    @Override public void voar() { ... }
    // Não implementou decolar() e pousar() → ERRO de compilação!
}

// ■ Usar protected em método de interface → erro de compilação
// ■ Usar implements em interface → interfaces usam extends
```

Padrões de Projeto que usam Abstração

Template Method	Classe Abstrata	Define esqueleto do algoritmo; subclasses preenchem etapas
Abstract Factory	Classe Abstrata	Cria famílias de objetos via hierarquia abstrata
Strategy	Interface	Algoritmo intercambiável via interface (sort, comparator)
Observer	Interface	Listeners via interface (ActionListener, EventListener)
Command	Interface	Encapsula ação como Runnable/Callable
Decorator	Interface	Envolve interface para adicionar comportamento

Boas Práticas

Prefira interfaces	Mais flexíveis e desacopladas que classes abstratas para a maioria dos casos.
@Override sempre	O compilador verifica a assinatura — evita erros silenciosos de typo no nome.
Coding to Interface	Use o tipo da interface como variável — permite trocar implementação sem refatorar.
Nomeie como comportamento	Comparable, Flyable, Serializable — o nome deve indicar o que o tipo pode fazer.
@FunctionalInterface	Use ao criar interfaces funcionais — documenta a intenção e o compilador verifica.
default com moderação	Não exagere nos defaults — podem tornar a interface complexa e difícil de entender.
Documente o contrato	Use Javadoc na interface — deixe claro o que cada método deve fazer, suas pré/pós-condições.

Resumo Final

Abstração e generalização são a base para escrever código orientado a objetos de qualidade. Classes abstratas fornecem estado e implementação parcial compartilhada entre tipos relacionados. Interfaces definem contratos de comportamento que classes completamente diferentes podem adotar. O JDK 8 e JDK 9 modernizaram as interfaces com métodos default, static e private. Coding to an Interface é a prática que une tudo: escreva código que depende de abstrações, não de implementações concretas.